

Primality Testing

BASIC CHECK & OPTIMIZATION

The Core Approach

A composite number 'n' must have at least one divisor 'd' such that:

$$1 < d \leq \sqrt{n}$$

Naive: Check 2 to $n - 1 \rightarrow O(n)$

Optimized: Check 2 to $\sqrt{n} \rightarrow O(\sqrt{n})$

C++ Implementation

```
bool isPrime(int n) {  
    if(n <= 1) return false;  
    for(int i=2; i*i <= n; i++) {  
        if(n % i == 0) return false;  
    }  
    return true;  
}
```

Best for – single checks and small values of 'n' .

SIEVE OF ERATOSTHENES

Algorithm Logic

Efficiently list all primes up to 'N' by marking composite numbers.

- ⇒ Assume all numbers are initially prime.
- ⇒ For each prime p , mark all its multiples $p.k$ as composite. ($k > 1$)
- ⇒ **Optimization:** Start marking multiples from p^2 (smaller multiples are already handled by smaller primes).

Usage: Essential for precomputing primes up to 10^7 .

	2	3	4	5	6	7	8	9	10	Prime numbers
11	12	13	14	15	16	17	18	19	20	
21	22	23	24	25	26	27	28	29	30	
31	32	33	34	35	36	37	38	39	40	
41	42	43	44	45	46	47	48	49	50	
51	52	53	54	55	56	57	58	59	60	
61	62	63	64	65	66	67	68	69	70	
71	72	73	74	75	76	77	78	79	80	
81	82	83	84	85	86	87	88	89	90	
91	92	93	94	95	96	97	98	99	100	
101	102	103	104	105	106	107	108	109	110	
111	112	113	114	115	116	117	118	119	120	

SIEVE OF ERATOSTHENES

Algorithm Logic

Efficiently list all primes up to 'N' by marking composite numbers.

- ⇒ Assume all numbers are initially prime.
- ⇒ For each prime p , mark all its multiples $p.k$ as composite. ($k > 1$)
- ⇒ **Optimization:** Start marking multiples from p^2 (smaller multiples are already handled by smaller primes).

Usage: Essential for pre-computing primes up to 10^7

```
cpp

vector<int> sieve(int n) {
    vector<bool> isPrime(n+1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= n; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i) {
                isPrime[j] = false;
            }
        }
    }
    vector<int> primes;
    for (int i = 2; i <= n; i++) {
        if (isPrime[i]) primes.push_back(i);
    }
    return primes;
}
```

EFFICIENCY & APPLICATIONS

Complexity

Sum of prime reciprocals
leads to:

$O(N \log \log N)$

SPF Factorization

Modify sieve to store **Smallest
Prime Factor**.

Factorize any number in:
 $O(\log N)$

Speed Advantage

Precomputing 10M numbers
takes ~0.1s.
Vastly superior to checking each
number individually.

```
// SPF Tweak: instead of prime[j]=false  
if(spfa[j] == j) spfa[j] = i;
```

EFFICIENCY & APPLICATIONS

Algorithm Logic

Goal: Precompute for every number x the smallest prime p that divides x .

Initialization: Set $\text{spf}[i] = i$ for all i , meaning initially assume each number is prime.

Sieve Logic: Similar to normal sieve, iterate primes and mark multiples, but instead of marking composite, assign their smallest prime factor:

$\rightarrow (\text{spf}[j] == j) \text{ spf}[j] = i$

Primality Check:

A number x is prime $\rightarrow \text{spf}[x] == x$.

```
cpp

vector<int> spf; // smallest prime factor array

void buildSPF(int n) {
    spf.resize(n + 1);

    // initialize spf[i] = i
    for (int i = 1; i <= n; i++)
        spf[i] = i;

    for (int i = 2; i * i <= n; i++) {
        if (spf[i] == i) { // i is prime
            for (int j = i * i; j <= n; j += i) {
                if (spf[j] == j)
                    spf[j] = i;
            }
        }
    }
}
```

PROBABILISTIC TESTS & CHALLENGES

Fermat's Primality Test

Based on: $a^{p-1} \equiv 1 \pmod{p}$

- ⇒ Pick random a . If formula fails, p is composite.
- ⇒ Fails for **Carmichael Numbers** (e.g., 561, 1105, 1729).
- ⇒ Leads to Miller-Rabin and AKS tests.

The Challenge (HW)

Problem: Print all primes in range $[L, R]$.

$L, R \leq 10^{12}$, but $R-L \leq 10^6$.

Hint: Segmented Sieve

Standard sieve uses too much memory. Use a sieve for and shift the range.

Sieve methods are often faster than probabilistic tests for small N and range queries.

Binary and Bitwise Operations

Efficiency at the Bit Level: Essential for Competitive Programming

Goal: Constant-time optimizations

OPERATOR TRUTH TABLES

A	B	A & B	A B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Note: ~ (NOT) is unary and inverts all bits.

SHIFT OPERATOR

Left Shift (<<)

$a \ll b \rightarrow a * 2^b$

Example: $5 \ll 1 = 10$

$101 \ll 1 = 1010$

Right Shift (>>)

$a \gg b$ divides a by 2^b (integer division).

Example: $5 \gg 1 = 2$

$101 \gg 1 = 10$

111100010100010001

Set Bits

XOR: CP MAGIC

- ✔ **Self-Inverse:** $x \oplus x = 0$. Perfect for finding elements that appear once.
- ✔ **Identity:** $x \oplus 0 = x$. XORing with 0 preserves the value.
- ✔ **Commutativity:** $x \oplus y = y \oplus x$. Order doesn't matter.
- ✔ **Associativity:** $(x \oplus y) \oplus z = x \oplus (y \oplus z)$. Grouping doesn't matter.
- ⚡ **Swap Trick:** $a \oplus= b; b \oplus= a; a \oplus= b$; Swaps two numbers without extra space.

| HIGH SPEED BIT COUNTING

C++ Compiler Intrinsics

`__builtin_popcount(n)` → Count of set bits (1s).

`__builtin_clz(n)` → Count leading zeros (important for $\log_2$).

`__builtin_ctz(n)` → Count trailing zeros (finds lowest set bit).

`__builtin_parity(n)` → Returns 1 if popcount is odd.

BIT MANIPULATION

111100010100010001



zeroth bit

Set Bit

```
n | (1 << i)
```

Turns the i th bit
ON.

Clear Bit

```
n & ~(1 << i)
```

Turns the i -th bit
OFF.

Toggle Bit

```
n ^ (1 << i)
```

Flips the i -th bit.

Check Bit

```
(n >> i) & 1
```

Is the i -th bit ON?

Search for One's Complement and Two's Complement !!

CP BIT TRICKS

Goal	Expression	Logic
Check Parity	$n \& 1$	0 if even, 1 if odd
Power of 2	$(n \& (n-1))$ (n can't be zero)	Exactly one bit is set
Clear Leftmost Set Bit	$n \& (n - 1)$	Clears lowest set bit
Get Leftmost Set Bit	$n \& (-n)$	Isolates lowest set bit
Multiply by 2^k	$n \ll k$	Fast multiplication

BITMASKING FOR SUBSETS

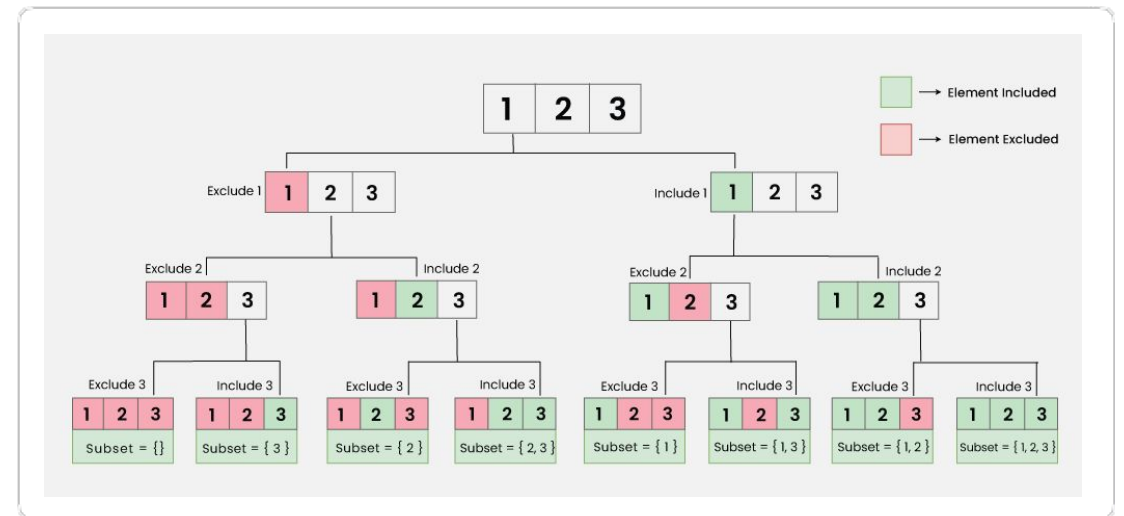
An integer can represent a set of up to 32 (or 64) elements. Each bit represents presence (1) or absence (0).

Generating all subsets:

```
for (int i = 0; i < (1 << N); ++i) {  
    for (int j = 0; j < N; ++j) {  
        if ((i >> j) & 1) {  
            // Element j is in subset i  
        }  
    }  
}
```

Complexity: $O(2^N)$

Essential for Dynamic Programming over subsets (Bitmask DP).



SUBSET SUM

Given numbers $a_1, a_2, \dots, a_n$, determine which sums can be formed using subset elements.

```
#include <bitset>
```

```
bitset<1000> dp; // bitset with 1000 bits (0 to 999)
```


EXAMPLE: SUBSETS OF {A, B, C}

Integer Value	Binary Representation	Subset Result
0	0 0 0	{ } (Empty Set)
1	0 0 1	{ A }
3	0 1 1	{ A, B }
7	1 1 1	{ A, B, C }

Application 3: XOR Efficiency

Simplifying Complex Logic with Exclusive-OR.

XOR PROPERTIES

Bit Flag

Store multiple boolean settings
(Read, Write, Exec) in a single
byte. Efficiently toggle states
with XOR.

BINARY EXPONENTIATION

| THE MATHEMATICAL LOGIC

Approach →

For any integer a and non-negative integer b :

$$a^b = \begin{cases} 1 & \text{if } b = 0 \\ (a^{\lfloor b/2 \rfloor})^2 & \text{if } b \text{ is even} \\ a \cdot (a^{\lfloor b/2 \rfloor})^2 & \text{if } b \text{ is odd} \end{cases}$$

This defines the recursive structure behind binary exponentiation.

Divide and Conquer: The exponent is halved in each step, reducing the number of multiplications.

Base Case: Any number raised to the power of 0 is 1.

Even/Odd Split: Odd exponents simply require one extra multiplication by the base.

| THE MATHEMATICAL LOGIC

More Correct Way to look at →

If we write b in binary:

$$b = \sum_{i=0}^k b_i 2^i \quad \text{where } b_i \in \{0, 1\}$$

Then:

$$a^b = \prod_{\substack{i=0 \\ b_i=1}}^k a^{2^i}$$

Example:

$$13 = (1101)_2 = 8 + 4 + 1$$

So:

$$a^{13} = a^8 \cdot a^4 \cdot a^1$$

| THE MATHEMATICAL LOGIC

More Correct Way to look at →

```
long long binpow(long long a, long long b) {  
    long long res = 1;  
    while (b > 0) {  
        if (b & 1)  
            res *= a;  
        a *= a;  
        b >>= 1;  
    }  
    return res;  
}
```

Modular Arithmetic & Prefix Sums

Basic Properties

+ Modular Addition:

$$(a + b) \% m = ((a \% m) + (b \% m)) \% m$$

- Modular Subtraction:

$$(a - b) \% m = ((a \% m) - (b \% m) + m) \% m$$

x Modular Multiplication:

$$(a \cdot b) \% m = ((a \% m) \cdot (b \% m)) \% m$$

Modular Multiplicative Inverse

Modular division means computing:

$$\frac{a}{b} \bmod m$$

But in modular arithmetic we cannot directly divide, because division is not defined like real numbers.

So modular division is actually defined as:

$$a \cdot b^{-1} \pmod{m}$$

where b^{-1} is the modular multiplicative inverse of b modulo m .

1. Using Fermat's Little Theorem (when m is prime)

If m is prime:

$$b^{-1} \equiv b^{m-2} \pmod{m}$$

So:

$$\frac{a}{b} \bmod m = a \cdot b^{m-2} \bmod m$$

Example:

$$\frac{10}{3} \bmod 7 = 10 \cdot 3^{7-2} \bmod 7 = 10 \cdot 3^5 \bmod 7$$

Prefix Sums

1D Prefix Sums

Precompute an array **P** where $P[i] = P[i-1] + A[i]$.

Index (i)	0	1	2	3	4	5
Array A	3	1	4	1	5	9
Prefix P	3	4	8	9	14	23

$$\text{Query Sum}(L, R) = P[R] - P[L-1]$$

2D Prefix Sums



Build

$$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + A[i][j]$$



Query

$$\text{Sum} = S[x2][y2] - S[x1-1][y2] - S[x2][y1-1] + S[x1-1][y1-1]$$

9	5	7	3
8	9	6	1
6	7	14	3
2	5	3	1

Questions?

THANK YOU FOR YOUR ATTENTION